

CS101: Data Structures and Algorithms — Page 1: Linear Data Structures and Asymptotic Foundations

1. Arrays and Dynamic Arrays

An array is a contiguous block of memory storing elements of homogenous type. In a static array, size is fixed at allocation time. Element access is achieved via pointer arithmetic: $\text{Address}(A[i]) = \text{BaseAddress} + i * \text{sizeof}(\text{Element})$, providing deterministic $O(1)$ time complexity.

Dynamic arrays (such as Python lists or C++ `std::vector`) automatically resize when capacity is reached. The common growth factor is 2. Resizing requires allocating a new array of double the capacity and copying all n elements, an $O(n)$ operation. However, amortized analysis over a sequence of n insertions shows that the amortized cost per insertion is $O(1)$.

2. Linked Lists

A linked list is a linear collection of data elements where order is not given by physical placement in memory. Instead, each element points to the next. In a Singly Linked List, each node contains a value and a 'next' pointer. In a Doubly Linked List, each node contains 'next' and 'prev' pointers.

- Insertion at the head is $O(1)$ time complexity.
- Insertion at arbitrary position requires traversal, taking $O(n)$ time.
- Search requires linear traversal, resulting in $O(n)$ worst-case time.
- Linked lists avoid contiguous memory requirements and prevent costly reallocation copying.

3. Stacks and Queues

A Stack operates on a Last-In, First-Out (LIFO) protocol. Key operations are push, pop, and peek, all executing in $O(1)$ time. Stacks are used in runtime call stacks, expression evaluation (Shunting-yard algorithm), and backtrack algorithms.

A Queue operates on a First-In, First-Out (FIFO) discipline. Operations include enqueue and dequeue in $O(1)$ time. Implementations frequently use circular buffers or doubly linked lists to avoid $O(n)$ shifting. Applications include breadth-first search and task scheduling buffers.

CS101: Data Structures and Algorithms — Page 2: Non-Linear Structures: Trees and Balancing

1. Binary Search Trees (BST)

A Binary Search Tree is a hierarchical structure where each node has at most two children. The BST invariant mandates that for any node X , all values in the left subtree are strictly less than X , and all values in the right subtree are strictly greater than X .

In an optimally balanced BST, search, insertion, and deletion operate in $O(\log n)$ time. However, if keys are inserted in sorted order, the BST degenerates into a linked list with height $O(n)$, causing worst-case operations to degrade to $O(n)$.

Traversals: In-order traversal (Left, Root, Right) of a BST outputs elements in monotonically ascending order. Pre-order is used for serialization, and Post-order is used for deletion and tree destruction.

2. Self-Balancing Trees: AVL and Red-Black Trees

To prevent tree degeneration, self-balancing binary search trees maintain logarithmic height.

- **AVL Trees:** For every node, the height difference between left and right subtrees (the Balance Factor) must be in $\{-1, 0, +1\}$. When an insertion or deletion violates this, tree rotations (Left Rotation, Right Rotation, Left-Right Rotation, or Right-Left Rotation) restore balance in $O(\log n)$ time.
- **Red-Black Trees:** Nodes are colored Red or Black. The root is Black, no two consecutive Red nodes are allowed, and every path from root to leaf contains the identical number of Black nodes. Red-Black trees require fewer rotations during mutations than AVL trees, making them standard for system library maps (such as C++ `std::map`).

CS101: Data Structures and Algorithms — Page 3: Hash Tables and Graph Algorithms

1. Hash Tables and Collision Resolution

A Hash Table maps keys to indices in an array using a hash function: $\text{index} = \text{hash}(\text{key}) \bmod \text{capacity}$. An effective hash function distributes keys uniformly across slots to minimize collisions.

Collision Resolution Strategies:

- Separate Chaining: Each bucket holds a linked list or small balanced tree of entries that hash to the same bucket. Average lookup time is $O(1 + \alpha)$, where $\alpha = n / m$ is the load factor.
- Open Addressing: All elements reside directly in the table. On collision, probe sequences are examined: Linear Probing ($\text{hash}(k) + i \bmod m$), Quadratic Probing ($\text{hash}(k) + c_1 \cdot i + c_2 \cdot i^2 \bmod m$), or Double Hashing ($\text{hash}_1(k) + i \cdot \text{hash}_2(k) \bmod m$).

2. Graph Representations and Traversal

A graph $G = (V, E)$ comprises vertices V and edges E . Graphs are represented using:

- Adjacency Matrix: A 2D array of size $|V| \times |V|$. Edge lookup is $O(1)$, but memory complexity is $O(|V|^2)$, inefficient for sparse graphs.
- Adjacency List: An array or dictionary of lists, storing adjacent neighbors for each vertex. Space complexity is $O(|V| + |E|)$.

Traversals:

- Breadth-First Search (BFS): Explores nodes level by level using a FIFO queue. Time complexity is $O(|V| + |E|)$. BFS identifies the shortest path in unweighted graphs.
- Depth-First Search (DFS): Explores branches deeply using recursion or a LIFO stack. Time complexity is $O(|V| + |E|)$. Used for cycle detection, topological sorting of DAGs, and strongly connected components.

CS101: Data Structures and Algorithms — Page 4: Asymptotic Complexity and Sorting Algorithms

1. Asymptotic Notations

Computational complexity assesses algorithm efficiency as input size n approaches infinity.

- Big-O (O): Asymptotic upper bound. $f(n) = O(g(n))$ if there exist constants $c > 0$ and n_0 such that $f(n) \leq c * g(n)$ for all $n \geq n_0$.
- Big-Omega (Ω): Asymptotic lower bound. $f(n) = \Omega(g(n))$ if $f(n) \geq c * g(n)$ for all $n \geq n_0$.
- Big-Theta (Θ): Asymptotic tight bound. $f(n) = \Theta(g(n))$ if $f(n)$ is both $O(g(n))$ and $\Omega(g(n))$.

2. Comparison Sorting Algorithms

- Quicksort: Divide-and-conquer algorithm. Selects a pivot, partitions array into elements less than and greater than pivot, and recursively sorts subarrays. Average time complexity is $O(n \log n)$. Worst-case time is $O(n^2)$ when pivot is poorly chosen (e.g., smallest/largest in already sorted array). Mitigated by Randomized Pivot selection.
- Mergesort: Divides array into halves, recursively sorts, and merges sorted halves. Guaranteed $O(n \log n)$ worst-case time complexity. Stable sort, but requires $O(n)$ auxiliary space.
- Heapsort: Builds a max-heap in $O(n)$ time, then repeatedly extracts maximum element in $O(\log n)$ time. In-place sorting with $O(n \log n)$ worst-case time complexity, but not stable.

3. Binary Search

Binary Search locates a target element in a sorted array by repeatedly halving the search interval. At each step, the target is compared with the median element.

Recurrence relation: $T(n) = T(n/2) + O(1)$. By the Master Theorem, time complexity is $O(\log n)$. Space complexity is $O(1)$ iterative or $O(\log n)$ recursive.